

# Practice Problem Part I — Non-Linear Optimization

## Gradient Inversion Attack in Federated Learning

Non-Linear Optimization Course

June 10, 2025

### Background

Federated Learning (FL) keeps training data on client devices and shares only gradients  $\mathbf{g} = \nabla_{\theta} \mathcal{L}(f_{\theta}(\mathbf{x}), y)$ . Yet, *gradient inversion* can reconstruct  $(\mathbf{x}, y)$  from  $\mathbf{g}$  and the current model  $f_{\theta}$  [1, 2, 3, 4].

### Optimization Problem

Given:

- model parameters  $\theta$  (2-layer MLP, trained on MNIST);
- a leaked gradient vector  $\mathbf{g}^*$  from a *single* unknown sample.

Recover an image  $\hat{\mathbf{x}} \in [0, 1]^{28 \times 28}$  and label  $\hat{y} \in \{0, \dots, 9\}$  by minimizing

$$\min_{\mathbf{x} \in [0, 1]^{784}, y \in \{0, \dots, 9\}} \left\| \nabla_{\theta} \mathcal{L}(f_{\theta}(\mathbf{x}), y) - \mathbf{g}^* \right\|_2^2 + \lambda \|\mathbf{x}\|_2^2.$$

### Tasks

1. Implement L-BFGS, Adam, or both, to optimize  $\mathbf{x}$  (continuous) and  $y$  (use one-hot logits or the iDLG pseudo-label trick).
2. Run at least five random initializations and keep the best solution.
3. Plot objective value over iterations and pixel MSE versus ground truth (provided for grading).
4. Study sensitivity to  $\lambda$  and optimizer hyper-parameters.

### Deliverables (100 pts)

- **Code (40 pts)** — reproducible, with `requirements.txt`.
- **Report (40 pts)** — max 4 pages: formulation, method, results, discussion.
- **Slides (20 pts)** — 5 min lightning talk.

## Starter Code (PyTorch)

All necessary functions are provided in `starter.py`. The script is well-commented to help you understand how to:

- Load and train a 2-layer MLP on MNIST.
- Simulate gradient leakage from a single or batched input.
- Reconstruct inputs by minimizing gradient difference.
- Visualize both the reconstructed image and optimization trajectory.

**Usage:** run `starter.py` directly:

```
python starter.py
```

This will:

1. Train a 2-layer MLP briefly on MNIST.
2. Simulate gradient leakage from a randomly selected image.
3. Reconstruct both a single image and a small batch.
4. Plot the reconstruction and the optimization loss over time.

You are encouraged to modify:

- The optimizer (e.g., try Adam instead of L-BFGS);
- The regularization parameter  $\lambda$ ;
- The number of restarts, learning rates, and initialization strategies.

**Functions available in `starter.py`:**

```
train_model(...)          # trains MLP
sample_single(...)        # draws one MNIST sample
sample_batch(...)         # draws a batch of samples
compute_gradient(...)     # computes gradient of loss wrt
                           parameters
reconstruct_single(...)   # reconstructs image+label from a
                           single g_star
reconstruct_batch(...)    # reconstructs batch from g_star
plot_reconstruction(...)  # shows ground truth vs. reconstructed
                           image
plot_loss(...)            # plots optimization trajectory
```

## Hints

- Normalise  $\mathbf{g}^*$  and use gradient clipping.
- Use multiple restarts—the landscape has many poor local minima.
- TV-loss or generative priors (GAN latents) can refine images but enlarge the search space.

## Ethics Notice

Apply this attack **only** to the classroom model and data.

## References

- [1] L. Zhu, Z. Liu, S. Han. *Deep Leakage from Gradients*. NeurIPS 2019.

- [2] L. Zhu, S. Han. *iDLG: Improved Deep Leakage from Gradients*. arXiv:2001.02610 (2020).
- [3] A. Ribeiro *et al.* *DLG-FB: Feedback-Blending Gradient Inversion*. arXiv:2409.17767 (2024).
- [4] P. Guo *et al.* *Survey of Gradient Inversion Attacks*. 2025.